

FALL 2025

COMP2211: Exploring Artificial Intelligence

Revision Notes

Hee Jae Cho

HKUST

Contents

1	Bayes' Theorem and the Naïve Bayes Algorithm	3
1.1	Building Intuition	3
1.2	Formalizing Bayes' Theorem	4
1.3	Generalized Bayes' Theorem	5
1.3.1	Multiple Beliefs	5
1.3.2	Multiple Beliefs and Multiple Pieces of Evidence	5
1.4	Introducing the Naïve Bayes	6
1.4.1	Formal Definition	6
1.4.2	Simplification	6
1.4.3	Increasing the Granularity of the Evidence	7
1.4.4	The Zero-Frequency Problem and α -Laplace Smoothing	8
1.4.5	Continuous Features in the Naïve Bayes	10
1.4.6	Probability Normalization	12
1.4.7	Underflow Prevention	12
1.5	Synthesis: Mixed Discrete and Continuous Naïve Bayes with Laplace Smoothing	12
2	K-Nearest Neighbors (KNN) Clustering	14
2.1	Process Overview	14
2.2	KNN Implementation	14
2.3	Data Standardization	16
2.4	Distance Functions	17
2.4.1	Manhattan Distance	17
2.4.2	Cosine Distance	17
2.4.3	Hamming Distance	18
2.5	Effective K selection	18
2.5.1	K Sensitivity	18
2.5.2	"Good" K Values	18
2.5.3	Rule of Thumb	18
2.6	K Selection via Cross Validation	19
2.6.1	Cross-Validation Approach	19
2.7	KNN Model Evaluation Methods	20
2.7.1	Binary Confusion Matrix	20
2.7.2	Precision, Recall, and F1 Score	20
2.7.3	Matthew's Correlation Coefficient	21
2.7.4	Multi-Class Confusion Matrices	21
2.7.5	Micro F1	21
2.7.6	Macro F1	22
2.7.7	Weighted F1	22
2.7.8	Error Measurement for Regression Problems	23
3	K-Means Clustering	24
3.1	K-Means Algorithm Steps	24
3.2	Mathematical Formulation	25
3.3	Distance Metrics	25
3.4	Practical Example	25
3.4.1	Iteration Details	26
3.5	Choosing the Optimal K	26
3.6	Advantages and Limitations	26

3.6.1	Advantages	26
3.6.2	Limitations	26
3.7	Best Practices	26
3.8	Python Implementation	27
3.9	Applications	27
3.10	Comparison with KNN	28
3.11	Advanced Considerations	28
4	Intro to ANNs: The Artificial Neuron	29
4.1	On the Nature of Classification	29
4.1.1	Examples of Classification	29
4.2	Artificial Neurons	29
4.2.1	What is an Artificial Neuron?	30
4.2.2	Components of an Artificial Neuron	30
4.2.3	Mathematical Formulation	31
4.3	Activation Functions	32
4.3.1	Sigmoid/Logistic Activation Function	32
4.3.2	Binary Step Activation Function (Perceptron)	32
4.4	Worked Example 1: Logical AND ($\wedge$)	33
4.4.1	Learning Process	33
4.4.2	When do we stop?	34
4.5	Interpreting Final Results: The Decision Boundry	34
4.6	Worked Example 2: Logical OR ($\vee$)	35
5	Multi-Layer Perceptrons (MLPs)	37
5.1	Building Intuition: Recognizing Hand-drawn Numbers	37
5.2	Intuition behind Hidden Layers	38
5.3	How to update the weights and biases?	40

1 Bayes' Theorem and the Naïve Bayes Algorithm

1.1 Building Intuition

Imagine we are introduced to a person named Bob, whom we describe as a **shy and organized person**. Is Bob more likely to be a **librarian or a farmer**?

Most people would say that he's more likely to be a librarian, but they fail to consider the relevance of the **ratio of librarians to farmers**.

Suppose we now have a sample of 200 farmers, and about 10 librarians (close to the real-life approximate ratio of farmers to librarians).

- Amongst the librarians, about 40% (4) fit Bob's description.
- Amongst the farmers, about 10% (20) match Bob's description.
- Even when giving the librarians a 40% match to Bob's description, the chances of Bob being a librarian are still only $\frac{1}{6}$, because $\frac{4}{4+20} = \frac{1}{6}$.

The above can be written as:

$$P(\text{Librarian}|E) = \frac{1}{6}$$

What we've just demonstrated is a simple version of Bayes' Theorem, and the idea that, at its core, Bayes' theorem tells us that we update our **Beliefs** based off of new **Evidence**. We use Bayes' Rule when:

- We are given a Hypothesis (**Belief/Class**) that we want to prove either true or false
- We are given a piece of new evidence (**Feature**)
- We are trying to find the probability of the Belief being true, given the evidence we have observed:

$$P(B|E)$$

B := The Belief (Hypothesis)

E := The Evidence we have observed

1.2 Formalizing Bayes' Theorem

We define relevant probabilities below:

$P(B)$:= The probability that the Belief is true in general (The Prior)

$P(\neg B)$:= The probability that the Belief is false (the complement of B).

$P(E|B)$:= Probability of Seeing the Evidence in the cases where Belief is true (Likelihood)

$P(E|\neg B)$:= Probability of Evidence given Belief is false

$P(B|E)$:= Probability of Belief being true given the Evidence (The Posterior Probability)

We finally define the posterior probability as:

$$\therefore P(B|E) = \frac{P(B)P(E|B)}{P(B)P(E|B) + P(\neg B)P(E|\neg B)}$$

In plain english, you can read the above as: *"The probability that the belief is true given the evidence we have observed is calculated by comparing the cases where both the belief and the evidence occur to all cases where the evidence occurs, whether or not the belief is true."*

Take note that the denominator for the above expression is essentially asking for the probability that the evidence is true in general (in all cases). Therefore we can further simplify Bayes' theorem to:

$$\therefore P(B|E) = \frac{P(B)P(E|B)}{P(E)}$$

Where

$P(E)$:= The Probability of the Evidence (The Marginal Probability)

We now perform the same analysis we did for the example with Bob by applying Bayes' Theorem:

$$\begin{aligned} P(\text{Librarian} | E) &= \frac{P(E | \text{Librarian}) \cdot P(\text{Librarian})}{P(E)} \\ &= \frac{0.4 \cdot \frac{10}{210}}{0.4 \cdot \frac{10}{210} + 0.1 \cdot \frac{200}{210}} \\ &= \frac{4/210}{4/210 + 20/210} \\ &= \frac{4/210}{24/210} \\ &= \frac{4}{24} \\ &= \frac{1}{6} \end{aligned}$$

1.3 Generalized Bayes' Theorem

1.3.1 Multiple Beliefs

What if for a given dataset, we had multiple possible beliefs? For example, instead of just testing for the belief that Bob is a farmer, we test if he's a farmer, a wrestler, or a doctor too. We now define Bayes' theorem with i beliefs as follows:

$$\begin{aligned} P(B_i|E) &= \frac{P(B_i)P(E|B_i)}{P(E|B_1)P(B_1) + P(E|B_2)P(B_2) + \dots + P(E|B_n)P(B_n)} \\ &= \frac{P(B_i)P(E|B_i)}{\sum_{j=1}^n P(B_j)P(E|B_j)} \end{aligned}$$

For example, if we were testing for 3 Beliefs, whether or not Bob was a doctor, librarian, or wrestler, we would define the posterior probabilities as the following:

$$P(L|E) = \frac{P(L)P(E|L)}{P(E|L)P(L) + P(E|W)P(W) + P(E|D)P(D)}$$

$$P(D|E) = \frac{P(D)P(E|D)}{P(E|L)P(L) + P(E|W)P(W) + P(E|D)P(D)}$$

$$P(W|E) = \frac{P(W)P(E|W)}{P(E|L)P(L) + P(E|W)P(W) + P(E|D)P(D)}$$

Where L, D, W each represent the belief that Bob is a Librarian, Doctor, and Wrestler, respectively.

1.3.2 Multiple Beliefs and Multiple Pieces of Evidence

If we were now asked to generalize Bayes' theorem for not only multiple possible Beliefs but also multiple pieces of Evidence, what would we do? It would be like asking for the theorem that tests for if Bob is a farmer, wrestler, or doctor, given the pieces of evidences that 1) he likes to read books, 2) he has completed med school, 3) he goes to the gym regularly. Assume that the set of d possible pieces of evidence is defined as:

$$E = \{e_1, e_2, e_3 \dots e_d\}$$

We now define Bayes' Theorem as the following:

$$\begin{aligned} P(B_i|E) &= \frac{P(B_i)P(E|B_i)}{P(E|B_1)P(B_1) + P(E|B_2)P(B_2) + \dots + P(E|B_n)P(B_n)} \\ &= \frac{P(B_i)P(E|B_i)}{\sum_{j=1}^n P(B_j)P((e_1, e_2, e_3 \dots e_d)|B_j)} \end{aligned}$$

1.4 Introducing the Naïve Bayes

1.4.1 Formal Definition

We define Bayes' theorem as being *naïve* when we assume that all pieces of evidence:

1. **Are independent of each other** (e.g., high temperature has nothing to do with high humidity)
2. **Contribute equally** to the probability that a test case belongs to a certain belief (class)

From this, we define the expression for the Naïve Bayes theorem as:

$$\begin{aligned} P(B_i | E) &= \frac{P(B_i) P((e_1, e_2, e_3 \dots e_d | B_i) | B_i)}{\sum_{j=1}^n P((e_1, e_2, e_3 \dots e_d | B_j) P(B_j))} \\ &= \frac{P(B_i) P(e_1 | B_i) P(e_2 | B_i) \dots P(e_d | B_i)}{\sum_{j=1}^n P(e_1 | B_j) P(e_2 | B_j) \dots P(e_d | B_j) P(B_j)} \\ &= \frac{P(B_i) \prod_{n=1}^d P(e_n | B_i)}{\sum_{j=1}^n P(B_j) \prod_{n=1}^d P(e_n | B_j)} \end{aligned}$$

We interpret the above as:

1. The probability of the *i*th Belief (B_i), given the set of Evidence (E) is given by... [$P(B_i|E)$]
2. The prior probability of the Belief ($P(B_i)$) multiplied by the product over all features (Likelihoods), $\prod_{n=1}^d P(e_n|B_i)$... [**Numerator**]
3. over the summation ($\sum$) of the prior probabilities of all cases ($P(B_j)$) multiplied by the product over all their respective features (Likelihoods), $\prod_{n=1}^d P(e_n|B_j)$ [**Denominator**]

1.4.2 Simplification

Furthermore, let us now say that we are only interested in obtaining the most likely prediction (class/belief) out of the set of all possible beliefs B_i . Logically, this would be the Belief, B_i which yields the largest number in the numerator (as this is a fraction over the range of all total possibilities). Note that for all posterior probability calculations, the set of all possibilities (i.e. the denominator/ $P(E)$) stays the same. Therefore, we may further simplify the resultant belief as given by the Naïve Bayes theorem as being:

$$\therefore B_{NB} = \operatorname{argmax}_{B_i} [P(B_i) \prod_{n=1}^d P(e_n | B_i)]$$

Where

- $B_{NB} :=$ The belief as given by Naïve Bayes
- $\operatorname{argmax}_{B_i} [P(B_i) \prod_{n=1}^d P(e_n | B_i)] :=$ Belief B_i that maximizes the enclosed expression

1.4.3 Increasing the Granularity of the Evidence

Let us now consider the case where a piece of evidence (e_j) from the set of evidence E has multiple possible values s.t. $e_j \in \{v_1, v_2, \dots, v_k\}$ where v_k represents the possible values for e_j . We now further modify the likelihood calculations as being:

$$\begin{aligned} P(e_j = v_k | B_i) &= \frac{P(e_j = v_k, B_i)}{P(B_i)} \\ &= \frac{\text{Count of } (e_j = v_k, B_i)}{\text{Count of } (B_i)} \end{aligned}$$

Essentially, what we mean by *increasing the granularity of the evidence* is that we are adding more possible values for what a certain feature may represent. For example, we may consider a feature (evidence) such as **Fever**. In real life, people don't simply have or not have fever (fever vs $\neg$ fever). Instead, people have varying **degrees** of fever (e.g. none, low, mild, high). When we say that a feature has high granularity, it means the number of categories it can possibly occupy is large. For example, consider the table below:

People with a Disease (Belief)	Fever
Y	High
N	None
Y	Mild
Y	Low
N	High

Table 1: Disease and Fever Data

If we wanted to get the sample of people with high fever who have the disease, it would be given by:

$$\begin{aligned} P(\text{Fever} = \text{High} | Y) &= \frac{P(\text{Fever} = \text{High}, Y)}{P(Y)} \\ &= \frac{\text{Count of } (\text{Fever} = \text{High}, Y)}{\text{Count of } (Y)} \\ &= \frac{1}{3} \end{aligned}$$

1.4.4 The Zero-Frequency Problem and α -Laplace Smoothing

Refer back to Table 1. If this were our training data, how would our Naïve Bayes model react if our test data had Disease = N and Fever = Mild? This specific combination of data points never occurred in our training data, and thus its probability would equate to zero ($P(\text{Fever} = \text{mild} | N) = 0$) (i.e. $\frac{0}{2}$).

We address the above issue (*Zero-Frequency*) with a technique known as α -Laplace Smoothing. Laplace smoothing prevents the occurrence of the Zero-Frequency problems by adding a set value (α) to **every likelihood feature for each Belief**. This is demonstrated in the following equation:

$$P(e_j = v_k | B_i) = \frac{\text{Count of } (e_j = v_k, B_i) + \alpha}{\text{Count of } (B_i) + m \cdot \alpha}$$

where

$$\begin{aligned} m &:= \text{Number of possible values for } e_j \\ \alpha &:= \text{usually 1 (For 1-Laplace Smoothing)} \end{aligned}$$

Note that:

- Laplace smoothing is applied **only to likelihood probabilities** $P(E_j = v_k | B_i)$, not to priors $P(B_i)$.
- Smoothing adds a small count (α , usually 1) to **every possible category of a feature**, including categories that occur in the dataset but **not with the current belief**.
- The denominator accounts for all categories: $\text{count}(B_i) + m \cdot \alpha$, where m = total number of possible values for that feature (i.e. not necessarily the m value of the zero-frequency affected feature).
- Categories that were **never seen in training data** are not automatically handled; they require either:
 - Extending m to include the new category, or
 - Mapping rare/unseen categories to an “Other” group.
- Features are assumed **conditionally independent**; each feature is smoothed independently using its own number of categories.
- Smoothing only affects other **Categorical/Discrete** features. Continuous features don't rely on smoothing for new data.

Let us now consider an updated data set with multiple features to see Laplace Smoothing in Action:

People with a Disease (Belief)	Fever	Sneezing
Y	High	Excessive
N	None	None
Y	Mild	Low
Y	Low	Mild
N	Low	Excessive

Table 2: Disease, Fever, and Sneezing Data

and suppose we have a patient presenting with the following symptoms:

Diagnosis	Fever	Sneezing
?	None	Mild

Table 3: Patient 6 Symptom Data

What is the prediction according to 1-Laplace modified Naïve Bayes?

Case 1: $B_i = Y$

$$\begin{aligned}
 P(B_i = Y \mid E) &= P(Y) P(\text{Fever}=\text{None} \mid Y) P(\text{Sneezing}=\text{Mild} \mid Y) \\
 &= \left(\frac{3}{5}\right) \left(\frac{0}{3}\right) \left(\frac{1}{3}\right)
 \end{aligned}$$

Apply 1-Laplace Smoothing to Zero-Frequency Feature

$$\begin{aligned}
 &= \left(\frac{3}{5}\right) \left(\frac{1}{7}\right) \left(\frac{2}{7}\right) \\
 &\approx 0.02449
 \end{aligned}$$

Case 2: $B_i = N$

$$\begin{aligned}
 P(B_i = N \mid E) &= P(N) P(\text{Fever}=\text{None} \mid N) P(\text{Sneezing}=\text{Mild} \mid N) \\
 &= \left(\frac{2}{5}\right) \left(\frac{1}{2}\right) \left(\frac{0}{2}\right)
 \end{aligned}$$

Apply 1-Laplace Smoothing to Zero-Frequency Feature

$$\begin{aligned}
 &= \left(\frac{2}{5}\right) \left(\frac{2}{6}\right) \left(\frac{1}{6}\right) \\
 &\approx 0.02222
 \end{aligned}$$

$0.02449 > 0.02222 \implies$ **Patient has disease** $\square$

1.4.5 Continuous Features in the Naïve Bayes

In the examples we have tackled thus far, we have seen features that may take on a **Discrete** number of possible values (recall: $e_j \in \{v_1, v_2, \dots, v_k\}$). However, what if a feature can take on a **Continuous** range of possible values? We give the probability of a likelihood (recall: $P(E|B)$) of a **Continuous** feature follows in the **Gaussian Distribution** as shown below:

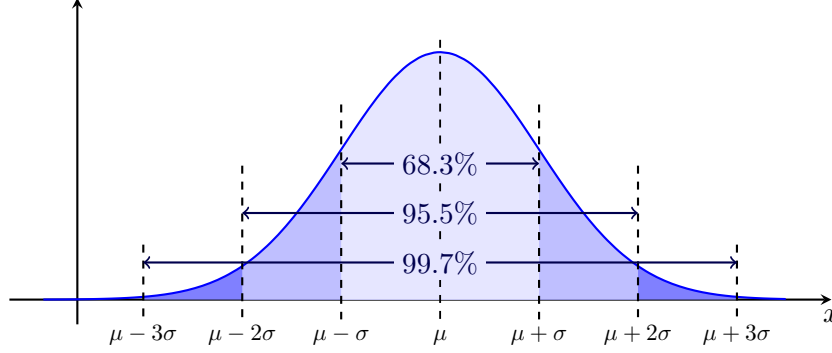


Figure 1: The Gaussian Distribution

From the above distribution, we deduce the likelihood's probability using the following equation:

$$\begin{aligned} f(x) &= \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \\ &= \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \end{aligned}$$

Where:

μ = Mean/Average of feature

$$= \frac{1}{n} \sum_{i=1}^n x^i$$

σ = Standard Deviation of Feature

$$= \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2}$$

For example, let us say that we tested patients for whether they contain a certain virus 'Z'. And we are given the following statistics about the data set:

Feature	Disease 'Z': Yes		Disease 'Z': No	
	Mean (μ)	SD (σ)	Mean (μ)	SD (σ)
Blood Pressure	73	6.2	75	7.9
Fever	79	10.2	86	9.7

Table 4: Gaussian Distribution Parameters for Disease 'Z'

Given the following patient data:

Diagnosis	Fever (°F)	Blood Pressure (mmHg)
?	97	66

Table 5: Patient Symptom Data

We aim to estimate the most likely diagnosis (“Z” or “¬Z”) using the Gaussian Naïve Bayes model.

Likelihood Calculation

For continuous features, the likelihood is modeled using a Gaussian (Normal) distribution:

$$P(x|B_i) = \frac{1}{\sqrt{2\pi}\sigma_i} e^{-\frac{(x-\mu_i)^2}{2\sigma_i^2}}$$

Case 1: Diagnosis = Z

$$P(BP = 66|Z) = \frac{1}{\sqrt{2\pi}(6.2)} e^{-\frac{(66-73)^2}{2(6.2)^2}} \approx \mathbf{0.0340}$$

$$P(F = 97|Z) = \frac{1}{\sqrt{2\pi}(10.2)} e^{-\frac{(97-79)^2}{2(10.2)^2}} \approx \mathbf{0.00824}$$

Case 2: Diagnosis = ¬Z

$$P(BP = 66|\neg Z) = \frac{1}{\sqrt{2\pi}(7.9)} e^{-\frac{(66-75)^2}{2(7.9)^2}} \approx \mathbf{0.0264}$$

$$P(F = 97|\neg Z) = \frac{1}{\sqrt{2\pi}(9.7)} e^{-\frac{(97-86)^2}{2(9.7)^2}} \approx \mathbf{0.0216}$$

Summary of Likelihoods

Diagnosis	$P(F = 97 B_i)$	$P(BP = 66 B_i)$
Z	0.00824	0.0340
¬Z	0.0216	0.0264

Table 6: Gaussian Likelihoods for Each Diagnosis

Note that the above values only represent the likelihoods of observing each symptom under each diagnosis. In order to determine the full Posterior calculation, we need to multiply the above values with any other likelihood calculations (both discrete and continuous) and multiply those, too, against the prior probability.

1.4.6 Probability Normalization

Note that when a problem asks you to “Normalize” a result, it simply means representing the probability of a belief as a ratio over the denominator, as we would originally do in the Naïve Bayes calculation. Essentially, it is asking for:

$$P(B|E) = \frac{P(B_i) \prod_{n=1}^d P(e_n | B_i)}{\sum_{j=1}^n P(B_j) \prod_{n=1}^d P(e_n | B_j)}$$

1.4.7 Underflow Prevention

As we’ve defined earlier:

$$B_{NB} = \operatorname{argmax}_{B_i} [P(B_i) \prod_{n=1}^d P(e_n | B_i)]$$

However, note that multiplying several probabilities between 0 and 1, by definition, can result in **floating-point underflow**. As such, using logarithms for these probabilities is often preferred in practice to prevent numerical underflow:

$$B_{NB} = \operatorname{argmax}_{B_i} [\log P(B_i) + \sum_{n=1}^d \log P(e_n | B_i)]$$

$$\because \log \prod_{n=1}^d x_n = \log x_1 + \log x_2 + \cdots + \log x_d$$

1.5 Synthesis: Mixed Discrete and Continuous Naïve Bayes with Laplace Smoothing

To consolidate our understanding of the Naïve Bayes algorithm, we now consider a dataset that includes:

- **Discrete binary features** (e.g., Cough: Yes/No)
- **Discrete categorical features** (e.g., Fever: None/Low/High)
- **Continuous features** (e.g., Body Temperature)

We aim to predict whether a person has a certain **Disease** ($D \in \{Y, N\}$).

Case	Disease (D)	Fever (Categorical)	Cough (Binary)	Temperature (°F)
1	Y	High	Yes	101.2
2	N	None	No	98.1
3	Y	Low	Yes	99.5
4	Y	Mild	No	100.3
5	N	Low	No	97.8
6	N	None	Yes	98.7

Table 7: Training Dataset: Mixed Discrete and Continuous Features

We now introduce a new test case, for which we must predict the most likely diagnosis. Observe that the value **Fever = Mild** under Disease = N never occurs in the training data — thus Laplace Smoothing is required to avoid the zero-frequency issue.

Case	Fever	Cough	Temperature (°F)
?	Mild	Yes	99.0

Table 8: Test Case

Case 1: $P(\text{Disease} = Y \mid E)$

$$P(\text{Disease} = Y \mid E) = P(Y) P(\text{Fever} = \text{Mild} \mid Y) P(\text{Cough} = \text{Yes} \mid Y) P(\text{Temp} = 99.0 \mid Y) \\ = \left(\frac{3}{6}\right) \left(\frac{1}{3}\right) \left(\frac{2}{3}\right) P(\text{Temp} = 99.0 \mid Y), \text{ Note likelihoods are unsmoothed}$$

Compute the mean and standard deviation for Temperature given Y :

$$\mu_Y = \frac{101.2 + 99.5 + 100.3}{3} \approx 100.33, \sigma_Y = \sqrt{\frac{0.7569 + 0.6889 + 0.0009}{2}} \approx 0.85$$

Gaussian likelihood for Temperature:

$$P(\text{Temp} = 99.0 \mid Y) = \frac{1}{\sqrt{2\pi}(0.85)} \exp\left(-\frac{(99.0 - 100.33)^2}{2(0.85)^2}\right) \approx 0.13799$$

Posterior probability with updated Laplace Smoothing:

$$P(\text{Disease} = Y \mid E) = \left(\frac{3}{6}\right) \left(\frac{2}{7}\right) \left(\frac{3}{5}\right) (0.13799) \approx \mathbf{0.0118277}$$

Case 2: $P(\text{Disease} = N \mid E)$

$$P(\text{Disease} = N \mid E) = P(N) P(\text{Fever} = \text{Mild} \mid N) P(\text{Cough} = \text{Yes} \mid N) P(\text{Temp} = 99.0 \mid N) \\ = \left(\frac{3}{6}\right) \left(\frac{0}{3}\right) \left(\frac{2}{5}\right) P(\text{Temp} = 99.0 \mid N)$$

Apply 1-Laplace smoothing for zero-frequency feature:

$$= \left(\frac{3}{6}\right) \left(\frac{1}{7}\right) \left(\frac{2}{5}\right) P(\text{Temp} = 99.0 \mid N)$$

Compute mean and standard deviation for Temperature given N :

$$\mu_N = \frac{98.1 + 97.8 + 98.7}{3} \approx 98.2, \sigma_N = \sqrt{\frac{0.01 + 0.16 + 0.25}{2}} \approx 0.458$$

Gaussian likelihood for Temperature:

$$P(\text{Temp} = 99.0 \mid N) = \frac{1}{\sqrt{2\pi}(0.458)} \exp\left(-\frac{(99.0 - 98.2)^2}{2(0.458)^2}\right) \approx 0.18946$$

Posterior probability:

$$P(\text{Disease} = N \mid E) = \left(\frac{3}{6}\right) \left(\frac{1}{7}\right) \left(\frac{2}{5}\right) (0.18946) \approx \mathbf{0.005413}$$

$\therefore 0.0118277 > 0.005413 \Rightarrow \mathbf{\text{Patient has the disease.}} \square$

2 K-Nearest Neighbors (KNN) Clustering

The K-Nearest Neighbors Clustering method is one of the most widely used supervised Machine Learning Algorithms, in due part, because of its simple implementation.

Furthermore, KNN is a type of lazy, **Non-Parametric**, ML algorithm. That is to say, the KNN algorithm makes no assumptions about the underlying form or distribution of the data provided (c.f. Naïve Bayes assume Gaussian distribution and independence).

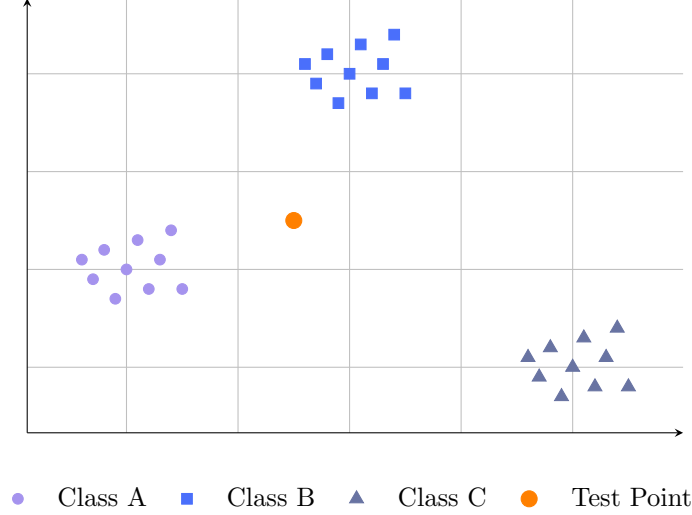


Figure 2: KNN Clustering

KNN relies on **labeled** data with several classes to predict the classification of the new sample point.

2.1 Process Overview

1. Prepare training and test data
2. Select K
3. Choose a distance function
4. Compute the distance of training data to n training data points
5. Sort computed distances and select the shortest K neighbors
6. Assign test data to class based on majority vote of the K selected neighbors

2.2 KNN Implementation

We define the set of training and test data as follows:

$$x^{\text{Train}} = \{x_1^{\text{Train}}, x_2^{\text{Train}}, \dots, x_n^{\text{Train}}\}$$
$$x^{\text{Test}} = \{x_1^{\text{Test}}, x_2^{\text{Test}}, \dots, x_n^{\text{Test}}\}$$

Note that both x^{Test} and x^{Train} have n features. In other words, if test data had the features Height and Weight, then the test data would also have these same features. Now, consider the sample training data table as presented below:

Height (in cm)	158	158	158	160	160	163	163	160	163
Weight (in kg)	58	59	63	59	60	60	61	64	64
T-shirt Size	M	M	M	M	M	M	M	L	L

Height (in cm)	165	165	165	168	168	168	170	170	170
Weight (in kg)	61	62	65	62	63	66	63	64	68
T-shirt Size	L	L	L	L	L	L	L	L	L

Table 9: T-shirt Size Training Data

Let us imagine that we are now presented with test data, whose class we must predict based off of the training data above. Let us introduce the training data as:

Shirt Size	Height (cm)	Weight (kg)
?	161	61

Table 10: Weight and Height Test Data

Now let us calculate the distances between the test data and each data point in our training data. For this example, let us use Euclidean Distance (we'll explore more distance functions later on):

$$\text{Euclidean Distance}(x^{Train}, x^{Test}) := \sqrt{\sum_{i=1}^n (x^{Train}_i - x^{Test}_i)^2}$$

In other words, Euclidean distance is defined as the square root of the summation of squared differences of each attribute i . Further simplifying, for the purposes of the above data set, we may imagine the Euclidean distance function as:

$$\text{Euclidean Distance} = \sqrt{(\text{Height}^{Train} - \text{Height}^{Test})^2 + (\text{Weight}^{Train} - \text{Weight}^{Test})^2}$$

Using the following method, we compute the following distances from the test data to each data point n of our training data:

Height (in cm)	158	158	158	160	160	163	163	160	163
Weight (in kg)	58	59	63	59	60	60	61	64	64
T-shirt Size	M	M	M	M	M	M	M	L	L
Distance between the training data and testing data	4.24	3.61	3.61	2.24	1.41	2.24	2.00	3.16	3.61

Height (in cm)	165	165	165	168	168	168	170	170	170
Weight (in kg)	61	62	65	62	63	66	63	64	68
T-shirt Size	L	L	L	L	L	L	L	L	L
Distance between the training data and testing data	4.00	4.12	5.66	7.07	7.28	8.60	9.22	9.49	11.40

Table 11: T-shirt Size Data and Calculated Distances

We now sort the n data points by shortest distances. Let us also assume that we had picked $K = 5$.

Height (in cm)	160	163	160	163	160	158	158	163	165
Weight (in kg)	60	61	59	60	64	59	63	64	61
T-shirt Size	M	M	M	M	L	M	M	L	L
Distance between the training data and testing data	1.41	2.00	2.24	2.24	3.16	3.61	3.61	3.61	4.00

Height (in cm)	165	158	165	168	168	168	170	170	170
Weight (in kg)	62	58	65	62	63	66	63	64	68
T-shirt Size	L	M	L	L	L	L	L	L	L
Distance between the training data and testing data	4.12	4.24	5.66	7.07	7.28	8.60	9.22	9.49	11.40

Table 12: Sorted T-shirt Size Data and Calculated Distances

The $K(5)$ shortest distances are then defined as the K data points with the shortest calculated distance, and we represent them below in gray:

Height (in cm)	160	163	160	163	160
Weight (in kg)	60	61	59	60	64
T-shirt Size	M	M	M	M	L
Distance between the training data and testing data	1.41	2.00	2.24	2.24	3.16

Table 13: K-Nearest Training Samples

Thus, by majority vote of the K nearest data points, we determine the class of the test data point as being M . Note that by **majority vote**, we don't mean a $> 50\%$ majority, but rather simply determine this value by the class with the greatest number of votes in general.

2.3 Data Standardization

Recall how we calculated Euclidean Distance earlier, note how height inadvertently had more “authority” in the distance calculation, as height is generally greater than weight. When using multiple units in distance calculations, we first **standardize** them so that they have equal say in the determination of the distance. We define the standardization function as follows:

$$X_{new} = \frac{X - \text{mean}}{\text{Standard Deviation}}$$

Where we define X as the attribute value, X_{new} as the standardized attribute value, mean as the mean attribute value across all training data, and standard deviation as the standard deviation of the attribute value across all training data.

Applying the standardized function to the previous ranking data, we now see that the K-nearest neighbors have changed (represented as the yellow distances):

Table 14: T-shirt Size Data and Standardized Distances (Color-swapped)

Height (in cm)	160	163	160	163	160	158	158	163	165
Weight (in kg)	60	61	59	60	64	59	63	64	61
T-shirt Size	M	M	M	M	L	M	M	L	L
Distance between the training data and testing data	1.41	2.00	2.24	2.24	3.16	3.61	3.61	3.61	4.00
Distance between the training data and testing data (standardization)	0.44	0.46	0.60	0.79	1.16	1.03	1.03	1.23	0.92

Height (in cm)	165	158	165	168	168	168	170	170	170
Weight (in kg)	62	58	65	62	63	66	63	64	68
T-shirt Size	L	M	L	L	L	L	L	L	L
Distance between the training data and testing data	4.12	4.24	5.66	7.07	7.28	8.60	9.22	9.49	11.40
Distance between the training data and testing data (standardization)	1.00	1.33	1.78	1.66	1.79	2.49	2.22	2.37	3.37

2.4 Distance Functions

When performing KNN clustering, we aren't just limited to the the Euclidean distance calculation method. Besides Euclidean distance, we also have several others defined below.

2.4.1 Manhattan Distance

Manhattan Distance, also known as City Block Distance, is represented as the summation over the absolute differences between training and test data points. Its formula is defined below as:

$$\text{Manhattan Distance}(x^{Train}, x^{Test}) := \sum_{i=1}^n |x^{Train}_i - x^{Test}_i|$$

As an example consider the case that: Training data: (Height, Weight) = (160, 60), Testing data: (Height, Weight) = (163, 61). Manhattan distance would be calculated as:

$$\text{Manhattan Distance}(x^{Train}, x^{Test}) = |160 - 163| + |60 - 61| = 4$$

2.4.2 Cosine Distance

Subsequently, cosine theta (used for determining Cosine Distance) is defined as follows:

$$\cos\theta = \frac{x^{Train} \cdot x^{Test}}{|x^{Train}| \cdot |x^{Test}|} = \frac{\sum_{i=1}^n (x^{Train}_i \cdot x^{Test}_i)}{\sqrt{\sum_{i=1}^n (x^{Train}_i)^2} \sqrt{\sum_{i=1}^n (x^{Test}_i)^2}}$$

Note that taking the square then subsequently the square root is functionally the same as taking the absolute value.

Finally, Cosine distance is defined as the complement of cosine theta:

$$\text{Cosine Distance} := 1 - \cos\theta$$

For example, setting training data: (Height, Weight) = (160, 60), and testing data: (Height, Weight) = (163, 61), we obtain:

$$\begin{aligned} \cos\theta &= \frac{\sum_{i=1}^n (x^{\text{Train}} \cdot x^{\text{Test}})}{\sqrt{\sum_{i=1}^n (x^{\text{Train}})^2} \sqrt{\sum_{i=1}^n (x^{\text{Test}})^2}} \\ &= \frac{(160 \cdot 163) + (60 \cdot 61)}{\sqrt{(160)^2 + (60)^2} \sqrt{(163)^2 + (61)^2}} \\ &\approx 0.99999977 \\ \text{Cosine distance} &= 1 - 0.99999977 \\ &\approx 2.26 \times 10^{-7} \end{aligned}$$

2.4.3 Hamming Distance

Finally, we define Hamming Distance as simply the number of bit positions in which the two bits are different while comparing two binary strings of equal length. Hamming distance is a way for **comparing two binary data strings**. For example, suppose there are two strings 1101 1001 and 1001 1101, then the hamming distance between the two strings is 2. This is illustrated further below:

1	1	0	1	1	0	0	1
1	0	0	1	1	1	0	1

Figure 3: Example Binary Strings with Hamming Distance 2

Generally, hamming distance is applicable to categorical data problems.

2.5 Effective K selection

2.5.1 K Sensitivity

As a general rule, lower values of K tend to be more sensitive to outliers, whereas higher values of K are more resilient to outliers as they consider more voters to decide the final classification.

2.5.2 “Good” K Values

- **Breaking Ties:** Two Methods

1. If the nearest K neighbors vote in a tie, simply decrement K until any one class wins in majority.
2. Put more weight on nearest points as opposed to farther points.

- Ideal K value is usually neither too large nor too small

2.5.3 Rule of Thumb

A good rule of thumb for approximating adequate K for sample size of n is to set:

$$K = \sqrt{n}$$

2.6 K Selection via Cross Validation

Selecting an appropriate value for K is crucial for KNN performance. A poor choice of K can lead to overfitting or underfitting.

2.6.1 Cross-Validation Approach

Suppose we wanted to select the most appropriate K value for a data set. We can achieve this using what's called the d -fold cross validation. The inputs of the algorithm are as follows:

1. A KNN Model
2. List of possible K values where $K \in \{K_1, K_2, K_3, \dots, K_q\}$
3. Training data with n data points
4. d number of "folds" (i.e. subsets of training data of approximately equal size)

The final output of d-fold cross validation is the best possible (lowest error) value for K . The full process is as follows:

1. Split the training data into d folds of approximately equal size
2. $\forall K \in \{K_1, K_2, \dots, K_q\}$:
 - (a) $\forall$ fold $i \in \{1, 2, 3, \dots, d\}$:
 - Fix fold i as **Validation set**
 - Train on remaining $d - 1$ folds (**Training Data**)
 - Classify each data point in the validation set using KNN trained from remaining $d - 1$ training data.
 - Record error E_i
 - (b) Compute average error: $E = \frac{1}{d} \sum_{i=1}^d E_i$
3. Select the K value with the lowest average error

Note that the above process is essentially a doubly-nested *for loop*. The process is further illustrated below:

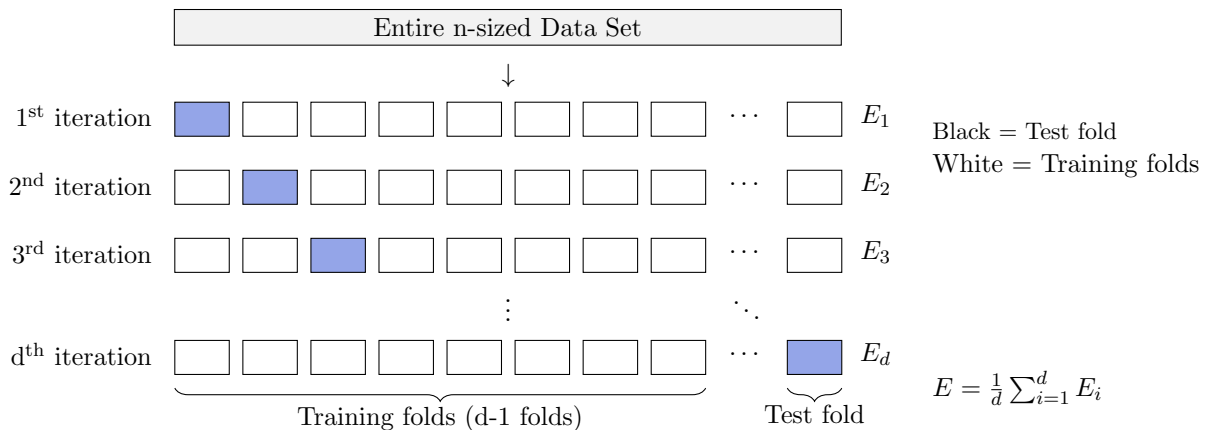


Figure 4: Cross Validation across d folds

2.7 KNN Model Evaluation Methods

2.7.1 Binary Confusion Matrix

For binary classification problems, the confusion matrix provides a comprehensive view of model performance:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Table 15: Binary Confusion Matrix

- **True Positive (TP)**: Positive cases that were correctly predicted positive
- **False Positive (FP)**: Negative cases incorrectly predicted positive class (Type I error)
- **True Negative (TN)**: Negative cases correctly predicted negative class
- **False Negative (FN)**: Positive cases incorrectly predicted negative class (Type II error)

Subsequently, the definitions for error and accuracy are defined as:

$$\begin{aligned}\mathbf{Error} &= \frac{FP + FN}{FP + FN + TP + TN} \\ &= 1 - \text{Accuracy}\end{aligned}$$

$$\begin{aligned}\mathbf{Accuracy} &= \frac{TP + TN}{FP + FN + TP + TN} \\ &= 1 - \text{Error}\end{aligned}$$

Essentially, error represents all the false predictions out of the set of all predictions, and accuracy represents all true predictions out of the set of all predictions.

2.7.2 Precision, Recall, and F1 Score

Precision Measures the accuracy of positive predictions:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall Measures the ability to find all positive samples:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score Harmonic mean of precision and recall:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

In an ideal world, we want a model that has a precision of 1 and a recall of 1. That means a F1-score of 1, i.e., a 100% accuracy which is often not the case for a machine learning model.

Trade-off Considerations

- **High Precision, Low Recall:** Conservative model, few false positives but misses many positives
- **Low Precision, High Recall:** Liberal model, finds most positives but has many false positives

2.7.3 Matthew's Correlation Coefficient

Matthew's Correlation Coefficient, also simply called MCC, provides a balanced measure even when classes are of very different sizes:

$$\text{MCC} = \frac{TN \times TP - FN \times FP}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Range of possible outputs: $[-1, +1]$

- $+1$: Perfect prediction
- 0 : Random prediction
- -1 : Total disagreement between prediction and actual

2.7.4 Multi-Class Confusion Matrices

For multi-class classification with C classes, the confusion matrix is $C \times C$:

	Pred. Class 1	Pred. Class 2	Pred. Class 3	Pred. Class C
Actual Class 1	TP_1	E_{12}	E_{13}	E_{1C}
Actual Class 2	E_{21}	TP_2	E_{23}	E_{2C}
Actual Class 3	E_{31}	E_{32}	TP_3	E_{3C}
Actual Class C	E_{C1}	E_{C2}	E_{C3}	TP_C

Table 16: Multi-class Confusion Matrix

For each class i :

$$TP_i = \text{True Positives for class } i$$

$$FP_i = \sum_{j \neq i} E_{ji} \quad (\text{False Positives})$$

$$FN_i = \sum_{j \neq i} E_{ij} \quad (\text{False Negatives})$$

$$TN_i = \sum_{j \neq i} \sum_{k \neq i} E_{jk} \quad (\text{True Negatives})$$

2.7.5 Micro F1

Micro-averaged F1 calculates metrics globally by aggregating all classes:

$$\begin{aligned}
\text{Total TP} &= \sum_{i=1}^C TP_i \\
\text{Total FP} &= \sum_{i=1}^C FP_i \\
\text{Total FN} &= \sum_{i=1}^C FN_i \\
\text{Micro Precision} &= \frac{\text{Total TP}}{\text{Total TP} + \text{Total FP}} \\
\text{Micro Recall} &= \frac{\text{Total TP}}{\text{Total TP} + \text{Total FN}} \\
\text{Micro } F_1 &= \frac{2 \times \text{Total TP}}{2 \times \text{Total TP} + \text{Total FP} + \text{Total FN}}
\end{aligned}$$

Note: Micro F1 equals overall accuracy in multi-class classification.

2.7.6 Macro F1

Macro-averaged F1 computes metrics for each class independently and averages them:

$$\begin{aligned}
\text{Macro Precision} &= \frac{1}{C} \sum_{i=1}^C \text{Precision}_i \\
\text{Macro Recall} &= \frac{1}{C} \sum_{i=1}^C \text{Recall}_i \\
\text{Macro } F_1 &= \frac{1}{C} \sum_{i=1}^C F_{1_i}
\end{aligned}$$

This approach treats all classes equally, regardless of their size.

2.7.7 Weighted F1

Weighted F1 accounts for class imbalance by weighting each class's contribution by its support (number of true instances):

$$\text{Weighted } F_1 = \frac{\sum_{i=1}^C (F_{1_i} \times \text{support}_i)}{\sum_{i=1}^C \text{support}_i}$$

Where support_i is the number of true instances for class i .

2.7.8 Error Measurement for Regression Problems

For KNN regression, common error metrics include:

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |a_i - p_i|$$

Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (a_i - p_i)^2$$

Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (a_i - p_i)^2}$$

Mean Absolute Percentage Error (MAPE)

$$\text{MAPE} = \frac{100\%}{m} \sum_{i=1}^m \left| \frac{a_i - p_i}{a_i} \right|$$

Where:

- a_i : Actual value for sample i
- p_i : Predicted value for sample i
- m : Number of samples

Comparison of Regression Metrics

- **MAE**: Robust to outliers, interpretable
- **MSE/RMSE**: Penalizes large errors more heavily
- **MAPE**: Scale-independent, good for relative error assessment

3 K-Means Clustering

Similar to KNN, K-Means is also a clustering ML method, however the process by which it determines clusters is different from KNN. At its core “ K ” in the context of K-Means, refers not to “ K -Neighbors” as it would in KNN clustering, but rather K “*Seeds*” which act as initial starting points for our future clusters. Note that the number of clusters is the same as K (i.e. the number of seed data points) To begin, consider the sample **unlabelled** data set below:

Person	Age	Income (\$000)	Expense Score
1	19	15	39
2	67	19	14
3	35	24	35
4	60	30	4
5	65	38	35
6	49	42	52
7	70	46	56
8	70	49	55
9	57	54	51
10	68	59	55
11	23	62	41
12	65	63	52
13	27	67	56
14	47	71	9
15	57	75	5
16	43	78	17
17	56	79	35
18	40	87	13
19	37	97	32
20	34	103	23

Table 17: Personal Financial Data - Clean Format

Is there a method for us to determine certain clusters that naturally fit within the data provided? Note that because the data is unlabelled, the method required to classify this data is a kind of **unsupervised** machine-learning algorithm. The end result of K-Means clustering is a clustered data set of naturally similar data points. Therefore, K-Means clustering may actually be used as pre-processing for further ML-Analysis (e.g. adding labels to data for the purposed of supervised learning).

3.1 K-Means Algorithm Steps

The K-Means algorithm follows an iterative process to partition data into K clusters:

1. **Initialize Centroids:** Randomly select K data points as initial centroids
2. **Assign Points:** Calculate distances between each data point and all centroids, assign each point to the nearest centroid
3. **Update Centroids:** Recompute centroids as the mean of all points in each cluster
4. **Repeat:** Continue steps 2-3 until convergence (no reassignments or minimal centroid movement)

3.2 Mathematical Formulation

Let the dataset be represented as:

$$D = \{x_1, x_2, x_3, \dots, x_n\}$$

where $x_i = (x_{i1}, x_{i2}, \dots, x_{id})$ is the i^{th} data point with d dimensions.

The algorithm minimizes the within-cluster sum of squares (WCSS):

$$\text{WCSS} = \sum_{j=1}^k \sum_{x \in C_j} \|x - \mu_j\|^2$$

where:

- C_j is the j^{th} cluster
- μ_j is the centroid of cluster C_j
- $\|x - \mu_j\|$ is the Euclidean distance between point x and centroid μ_j

3.3 Distance Metrics

While Euclidean distance is most commonly used, other distance metrics can be employed:

Euclidean Distance

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Manhattan Distance

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

Cosine Distance

$$d(x, y) = 1 - \frac{x \cdot y}{\|x\| \|y\|}$$

3.4 Practical Example

Let's demonstrate K-Means with a simplified example using the first 4 data points:

Person	Age	Income	Expense	Cluster
1	19	15	39	2
2	67	19	14	1
3	35	24	35	1
4	60	30	4	1

Table 18: K-Means Clustering Result (K=2)

3.4.1 Iteration Details

Initial Centroids: (67, 19, 14) and (19, 15, 39)

Distance Calculations:

$$d(\text{Point 1, Centroid 1}) = \sqrt{(19 - 67)^2 + (15 - 19)^2 + (39 - 14)^2} = 54.08$$

$$d(\text{Point 1, Centroid 2}) = \sqrt{(19 - 19)^2 + (15 - 15)^2 + (39 - 39)^2} = 0$$

$$d(\text{Point 3, Centroid 1}) = \sqrt{(35 - 67)^2 + (24 - 19)^2 + (35 - 14)^2} = 38.21$$

$$d(\text{Point 3, Centroid 2}) = \sqrt{(35 - 19)^2 + (24 - 15)^2 + (35 - 39)^2} = 18.79$$

3.5 Choosing the Optimal K

Selecting the right number of clusters is crucial. Common methods include:

Elbow Method Plot WCSS against different K values and look for the "elbow" point where the rate of decrease sharply changes.

Silhouette Analysis Measures how similar an object is to its own cluster compared to other clusters. Values range from -1 to 1, with higher values indicating better clustering.

3.6 Advantages and Limitations

3.6.1 Advantages

- **Simple and intuitive:** Easy to understand and implement
- **Efficient:** Scales well to large datasets
- **Flexible:** Can use various distance metrics
- **Guaranteed convergence:** Will always find a local optimum

3.6.2 Limitations

- **Sensitive to initial centroids:** Different initializations can yield different results
- **Requires specifying K:** Number of clusters must be predetermined
- **Sensitive to outliers:** Extreme values can distort cluster formation
- **Assumes spherical clusters:** Performs poorly with non-convex cluster shapes
- **Scale-dependent:** Features should be standardized for meaningful results

3.7 Best Practices

Data Preprocessing

- Standardize features to have zero mean and unit variance
- Handle missing values appropriately
- Remove or treat outliers that may distort clusters

Initialization Strategies

- **K-Means++:** Smart initialization that spreads initial centroids
- **Multiple runs:** Run algorithm multiple times with different random seeds
- **PCA preprocessing:** Reduce dimensionality before clustering

Convergence Criteria

- Maximum number of iterations reached
- Centroids stabilize (minimal movement between iterations)
- Cluster assignments stop changing
- WCSS improvement falls below threshold

3.8 Python Implementation

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import numpy as np

# Prepare data
data = np.array([[19,15,39], [67,19,14], [35,24,35],
                 [60,30,4], [65,38,35], [49,42,52]])

# Standardize features
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

# Apply K-Means
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(data_scaled)
centroids = kmeans.cluster_centers_

print("Cluster assignments:", clusters)
print("Centroids:", centroids)
```

3.9 Applications

K-Means clustering has diverse applications across domains:

- **Customer Segmentation:** Group customers by purchasing behavior
- **Image Compression:** Reduce color space by clustering similar colors
- **Document Clustering:** Group similar documents for topic modeling
- **Anomaly Detection:** Identify outliers as points far from any centroid
- **Market Research:** Segment markets based on demographic and behavioral data

3.10 Comparison with KNN

K-Means Clustering	K-Nearest Neighbors
Unsupervised learning	Supervised learning
No labels required	Requires labeled training data
Discovers natural groupings	Classifies based on similarity
Creates clusters	Finds nearest neighbors
Centroids represent clusters	No model, stores all data
Training: Find optimal centroids	Training: Store feature vectors
Prediction: Assign to nearest centroid	Prediction: Majority vote of neighbors

Table 19: K-Means vs KNN Comparison

3.11 Advanced Considerations

Dealing with Categorical Data For mixed data types, consider:

- K-Modes algorithm for categorical data
- Gower's distance for mixed data types
- One-hot encoding with careful distance computation

Validation Metrics

- **Silhouette Score:** Measures cohesion and separation
- **Calinski-Harabasz Index:** Ratio of between to within cluster dispersion
- **Davies-Bouldin Index:** Average similarity between clusters

Scaling to Large Datasets

- Mini-Batch K-Means for large datasets
- Parallel implementation using multiple cores
- Sampling techniques for very large datasets

4 Intro to ANNs: The Artificial Neuron

4.1 On the Nature of Classification

Classification refers to the process of determining a test case's class by comparing its features against training data. Classes represent labels or categories that we want to assign to inputs.

4.1.1 Examples of Classification

- **Spam Recognition:** 2 classes (Spam, Not Spam)
- **Hand-drawn Digit Recognition:** 10 classes (0, 1, 2, ..., 9)
- **Image Classification:** Multiple classes (e.g., cat, dog, neither)

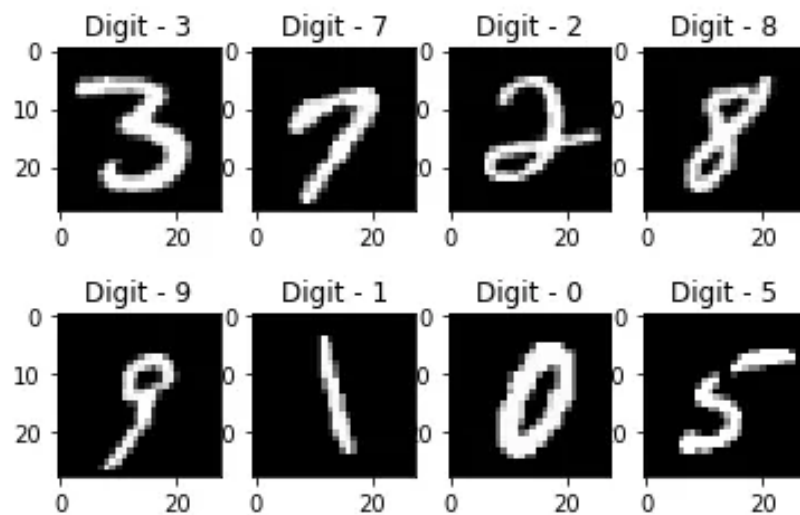


Figure 5: Classification of hand-drawn numbers

4.2 Artificial Neurons

Artificial Neural Networks (ANNs) are powerful AI algorithms known as **universal function approximators**. They transform inputs into outputs and draw inspiration from biological neurons.

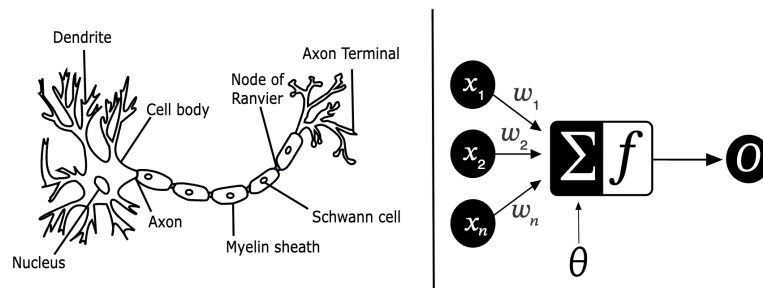


Figure 6: Biological vs. Artificial Neurons

Biological Neuron	Artificial Neuron
Dendrites	Inputs
Cell Nucleus	Node (Computation unit)
Axon	Output
Synapse	Weight

Table 20: Correspondence between biological and artificial neuron components

4.2.1 What is an Artificial Neuron?

- A simplified model of a biological neuron
- Performs calculations to detect patterns in input data
- The simplest model is the perceptron (Threshold Logic Unit)

4.2.2 Components of an Artificial Neuron

Each artificial neuron consists of two main components:

- **Linear Function:** Weighted sum of inputs plus bias

$$z = w_1x_1 + w_2x_2 + \theta$$

Note that the bias θ in other resources, might be denoted as simply b

- **Activation Function:** Determines whether the neuron should activate

$$\text{output} = f(z)$$

The activation function converts the weighted input into an output value that is fed to the next layer (**Hidden Layer**) or serves as the final output.

Let's now take a closer look at a single perceptron that receives 2 input values and understand its mathematical logic.

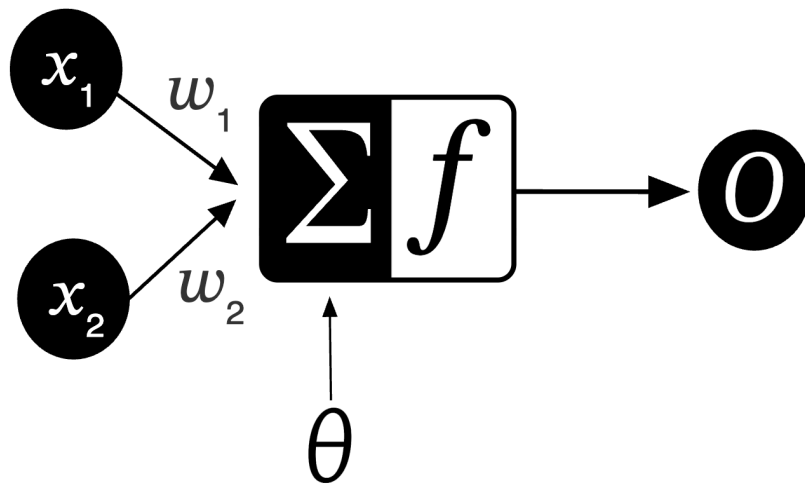


Figure 7: Close-up of an Artificial Neuron that Receives 2 Inputs

4.2.3 Mathematical Formulation

From Figure 7, we define:

$$\begin{aligned}x_1, x_2 &:= \text{Inputs} \\w_1, w_2 &:= \text{Weights} \\ \theta &:= \text{Bias} \\ f &:= \text{Activation Function} \\ \sum &:= \text{Summation of Weighted Inputs}\end{aligned}$$

The output is calculated as:

$$\text{output} = f(w_1 \cdot x_1 + w_2 \cdot x_2 + \theta)$$

Example Calculation: Given: $x_1 = 3$, $x_2 = 5$, $w_1 = 0.2$, $w_2 = 0.3$, $\theta = 0.3$,
 $f(x) = \frac{1}{1+e^{-x}}$

$$\begin{aligned}\text{output} &= f(0.2 \times 3 + 0.3 \times 5 + 0.3) \\ &= f(0.6 + 1.5 + 0.3) \\ &= f(2.4) \\ &= \frac{1}{1 + e^{-2.4}} \\ &= 0.91683\end{aligned}$$

4.3 Activation Functions

Activation functions determine the output of a neuron. The choice depends on the specific application requirements.

4.3.1 Sigmoid/Logistic Activation Function

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Takes any real number as input and outputs values between 0 and 1
- Larger inputs approach 1, smaller inputs approach 0
- Commonly used for probability outputs
- Smooth and differentiable

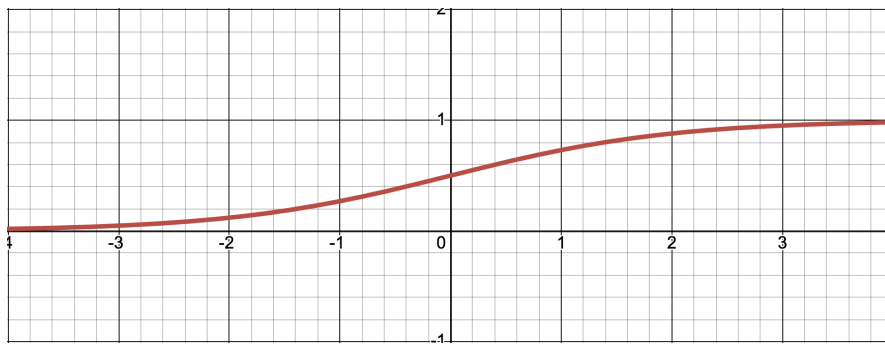


Figure 8: Sigmoid activation function

4.3.2 Binary Step Activation Function (Perceptron)

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

- Used in **perceptrons** or **threshold logic units**
- Provides binary outputs (0 or 1 only)
- Cannot handle multi-class problems
- Zero gradient hinders backpropagation in multilayer networks
- Suitable for representing logical operations
- Successor to the McCulloch-Pitts neuron model

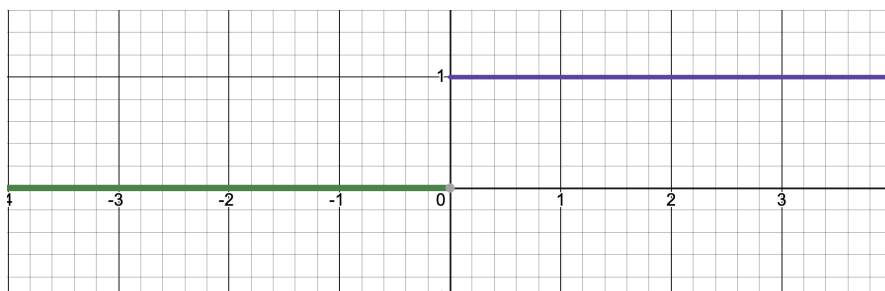


Figure 9: Binary step activation function

4.4 Worked Example 1: Logical AND ($\wedge$)

The logical AND operator has the following truth table:

x_1	x_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

Table 21: Logical AND Truth Table

For the purposes of this review, we may think of the above 4 rows of data as representing the entire training data set.

4.4.1 Learning Process

Initial parameters:

$$w_1 = 0.1, \quad w_2 = 0.5, \quad \theta = -0.8, \quad \eta = 0.2$$

Variable definitions:

$$\begin{aligned} x_1, x_2 &:= \text{Inputs} \\ \theta &:= \text{Bias} \\ w_1, w_2 &:= \text{Weights} \\ O &:= \text{Output} \\ T &:= \text{Target} \\ \Delta w_1, \Delta w_2 &:= \text{Weight changes} \\ \Delta \theta &:= \text{Bias change} \\ \eta &:= \text{Learning Rate} \end{aligned}$$

Learning Rules:

1. If the output is correct (i.e., T is the same as O), the weights w_1 and w_2 are not updated.
2. If the output is incorrect (i.e., T is different to O), the weights w_1 and w_2 are updated according to the following rules such that the output of the perceptron for the new weights is closer to T :

$$\begin{aligned} \Delta w_i &= \eta(T - O)x_i \\ \Delta \theta &= \eta(T - O) \\ w_i &= w_i + \Delta w_i \\ \theta &= \theta + \Delta \theta \end{aligned}$$

Recall that the output is calculated as:

$$O = f(w_1 \cdot x_1 + w_2 \cdot x_2 + \theta)$$

x_1	x_2	T	O	Δw_1	w_1	Δw_2	w_2	$\Delta \theta$	θ
-	-	-	-	-	0.1	-	0.5	-	-0.8
0	0	0	0	0	0.1	0	0.5	0	-0.8
0	1	0	0	0	0.1	0	0.5	0	-0.8
1	0	0	0	0	0.1	0	0.5	0	-0.8
1	1	1	0	0.2	0.3	0.2	0.7	0.2	-0.6
0	0	0	0	0	0.3	0	0.7	0	-0.6
0	1	0	1	0	0.3	-0.2	0.5	-0.2	-0.8
1	0	0	0	0	0.3	0	0.5	0	-0.8
1	1	1	1	0	0.3	0	0.5	0	-0.8

Table 22: Perceptron learning process for AND gate (highlighted rows show weight updates)

Each row represents one step in the learning process in which the perceptron adjusts its weights based on the error between predicted and target output (target output of the testing data may be thought of as the label). Every time the model completes one pass of the dataset this is considered one **Epoch**, and everytime the weights are updated, we call this **Learning**. By the end of the above proocess, the perceptron essentially learns the full logic of the AND expression.

4.4.2 When do we stop?

We have several methods for determining when we should stop training our perceptron model. Some methods include:

1. **Use the maximum training time:** The training may go a bit beyond the specified time limit in order to complete the current cycle.
2. **The maximum number of training cycles allowed:** If the maximum number of cycles is exceeded, then training stops.
3. **Use minimum accuracy:** Training will continue until the specified accuracy is attained.

4.5 Interpreting Final Results: The Decision Boundry

Refer back to Table 11, take note of the final weight values ($w_1 = 0.3, w_2 = 0.5, \theta = -0.8$, and recall that output is defined as $f(w_1 \cdot x_1 + w_2 \cdot x_2 + \theta)$. Based off of the above data, it would then be implied that:

$$\text{Output} = 0.3x_1 + 0.5x_2 - 0.8$$

Recall that the output is evaluated against the activation function where:

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

Therefore this may be rewritten as:

$$f(x) = \begin{cases} 0 & \text{if } 0.3x_1 + 0.5x_2 - 0.8 < 0 \\ 1 & \text{otherwise} \end{cases} = \begin{cases} 0 & \text{if } 3x_1 + 5x_2 < 8 \\ 1 & \text{otherwise} \end{cases}$$

Extrapolating further, we may now say that the shaded region defined below $3x_1 + 5x_2 < 8$ (which may be rewritten as $3x + 5y < 8$) are the x_1 and x_2 weight combinations which result in a 0 activation function output. The line which defines this region is known as the **Decision Boundry**. This relationship is illustrated in the graph below:

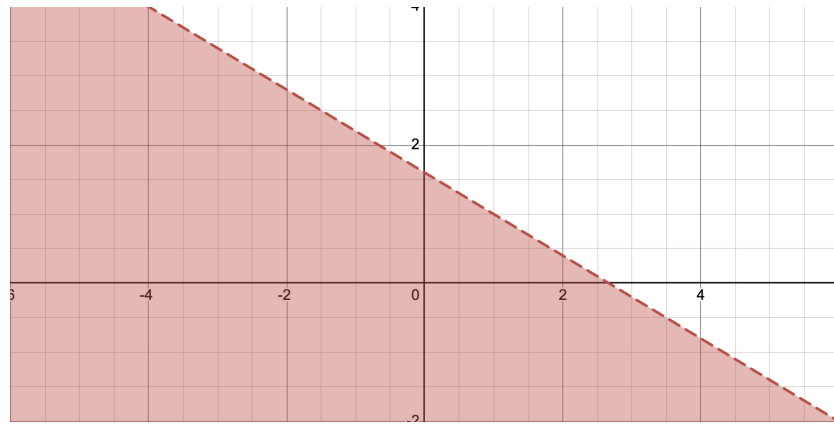


Figure 10: Decision Boundry Graph for AND operator training on a Perceptron

In other words, any combination of w_1 and w_2 that is graphed in the red-shaded region of the graph above produces an activation function signal of 0. In a binary classification problem, a decision boundary (or decision surface) is a hypersurface that partitions the underlying vector space into two sets, one for each class. The classifier will classify all the points on one side of the decision boundary as belonging to one class and all those on the other side as belonging to the other class. In this case, a binary output of either 0 or 1.

4.6 Worked Example 2: Logical OR ($\vee$)

Note the truth table for the logical OR operator below :

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

Table 23: Logical OR Truth Table

Assume that the weights and bias are randomly generated. For example, $w_1 = 0.1$, $w_2 = 0.5$, $\theta = 0.8$, $\eta = 0.2$. And perceptron activation function is defined as:

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

And output as:

$$O = f(w_1 \cdot x_1 + w_2 \cdot x_2 + \theta)$$

Determine the resultant decision boundary.

Based off of the initial starting conditions as defined above, we obtain the following learning process for the OR gate:

x_1	x_2	T	O	Δw_1	w_1	Δw_2	w_2	$\Delta \theta$	θ
-	-	-	-	-	0.1	-	0.5	-	-0.8
0	0	0	0	0	0.1	0	0.5	0	-0.8
0	1	1	0	0	0.1	0.2	0.7	0.2	-0.6
1	0	1	0	0.2	0.3	0	0.7	0.2	-0.4
1	1	1	1	0	0.3	0	0.7	0	-0.4
0	0	0	0	0	0.3	0	0.7	0	-0.4
0	1	1	1	0	0.3	0	0.7	0	-0.4
1	0	1	0	0.2	0.5	0	0.7	0.2	-0.2
1	1	1	1	0	0.5	0	0.7	0	-0.2
0	0	0	0	0	0.5	0	0.7	0	-0.2
0	1	1	1	0	0.5	0	0.7	0	-0.2
1	0	1	1	0	0.5	0	0.7	0	-0.2
1	1	1	1	0	0.5	0	0.7	0	-0.2

Table 24: Perceptron learning process for OR gate

Note how for the final epoch (i.e bottom 4 rows), we observe no changes in the weights. Generally, the training process for a simple perceptron is considered finished (or the network has **converged**) if the weights and bias remain unchanged for one entire pass through an epoch (the training data).

Convergence: If the weights do not update for the duration of a full epoch, it means the perceptron correctly classified every single data point in the training set during that epoch. Since the goal of training is to find a set of weights and a threshold (bias) that correctly separates the classes, the process is considered complete, and the training can be stopped.

Furthermore, as observed from the above table, the final weight and theta values are: $w_1 = 0.5$, $w_2 = 0.7$, and $\theta = -0.2$. Therefore we can now say that:

$$f(x) = \begin{cases} 0 & \text{if } 0.5x_1 + 0.7x_2 - 0.2 < 0 \\ 1 & \text{if } x \geq 0 \end{cases} = \begin{cases} 0 & \text{if } 5x_1 + 7x_2 < 2 \\ 1 & \text{if } x \geq 0 \end{cases}$$

As such, we illustrate the decision boundary graph as being the area under the line defined as $5x + 7y < 2$:

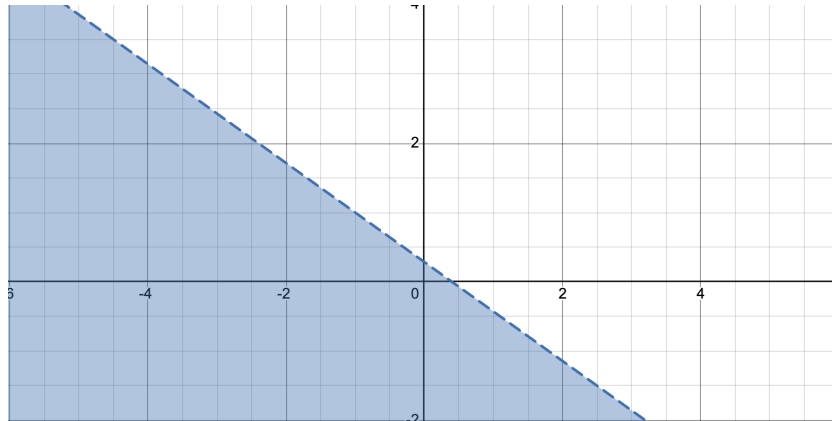


Figure 11: Decision Boundry Graph for OR gate

5 Multi-Layer Perceptrons (MLPs)

5.1 Building Intuition: Recognizing Hand-drawn Numbers

Lets imagine that we have a 28×28 grid of values ranging from 0 to 1. Let us define this number as the *luminosity* of that cell, where a value closer to 1 is white, and a value closer to 0 is dark. If people were to draw a single number on this 28×28 grid, a possible input for “5” might look like:

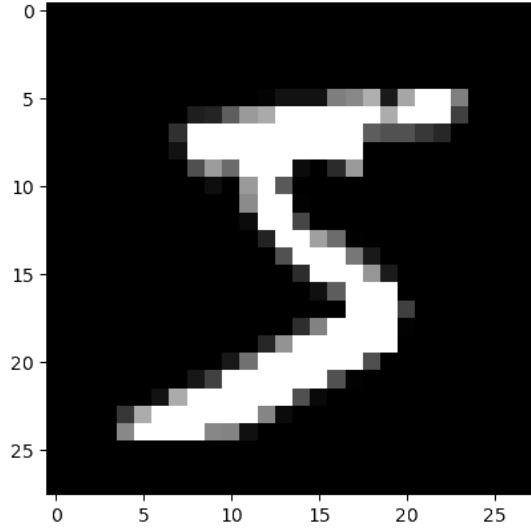


Figure 12: A 28 by 28 Matrix of Luminosity Values Depicting 5

If we imagined each pixel as being an input variable, we’d have a total of 784 inputs! If mapped this configuration to the original simple perceptron model we learnt about earlier, certainly there is no single combination of weights and biases that could correctly predict a set of 10 unique digits. This is where we introduce the concept of the Multi-layer Perceptron (MLP), and as the name suggests, it is essentially multiple artificial neurons layered together. The basic structure of an MLP neural network looks like this:

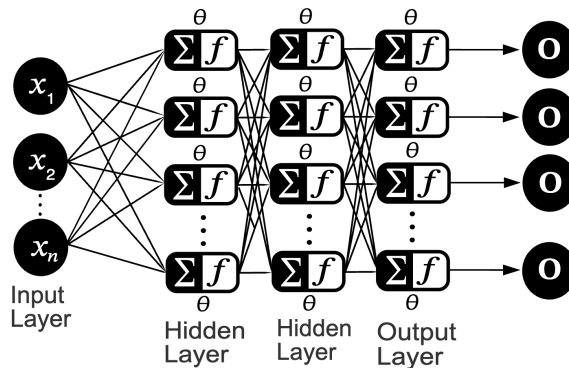


Figure 13: Multi-Layer Perceptron Structure

The first layer $(x_1, x_2, \dots, x_n)$ is called the **Input layer**. As mentioned earlier, this layer consists of the 784 inputs from the 28×28 grid, except that this time it is mapped

to multiple neurons instead of just one single perceptron. The rightmost layer is what is called the **Output layer**, and consists of 10 nodes representing each possible digit class. Between the output and input layer are the **Hidden Layers** whose primary purpose is deducing patterns from the inputs. For every node in the hidden and output layers, they have a summation and activation function, and each has a unique bias. Our full image recognition neural network might look something like this:

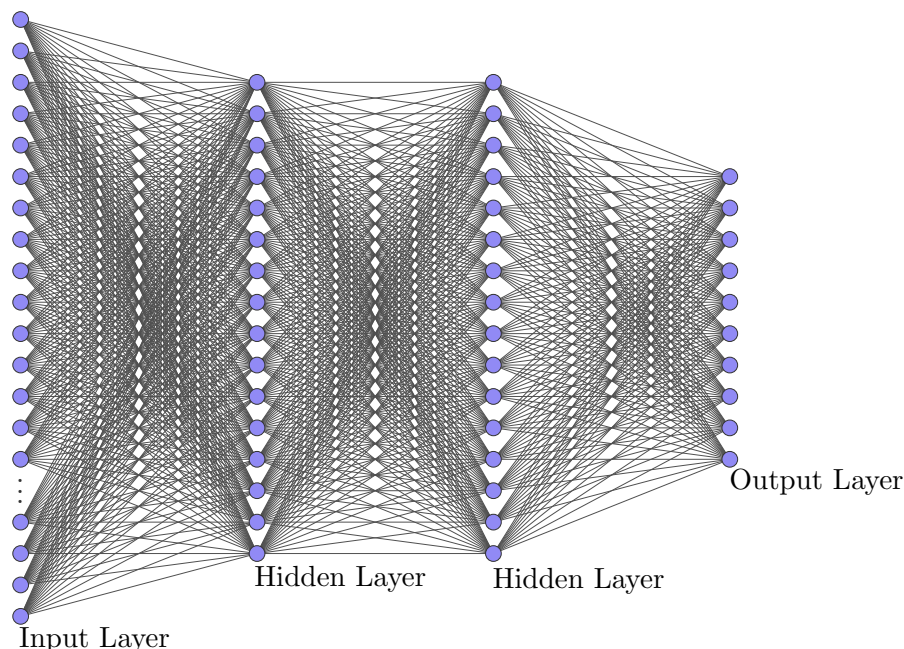


Figure 14: Neural Network of Digit Recognition

The specific structure of our neural network is as follows:

1. **Input Layer (784 Nodes)**: This layer represents the 784 pixel luminosity values of the hand-drawn digit.
2. **1st Hidden Layer (16 Nodes)**: Pattern recognition layer
3. **2nd Hidden Layer (16 Nodes)**: Pattern recognition layer
4. **Output Layer (10 Nodes)**: 10 possible digit guesses, where node with highest luminosity is the most likely guess.

From the above, it is relatively intuitive what the input and output layers do, but what exactly do the hidden layers do? We simply define the hidden layers as performing “Pattern-Recognition”, but how exactly does the neural network “learn” these patterns? What kind of patterns does it see?

5.2 Intuition behind Hidden Layers

When we say that the hidden layers recognize patterns in the input, ideally, we might imagine that they detect characteristics unique to each individual number. For example, the number ‘9’ might be defined by the patterns of having a loop on top and a line at the bottom. Or a number ‘8’ might be defined as having two loops on both the top and the bottom. This idea is illustrated below:

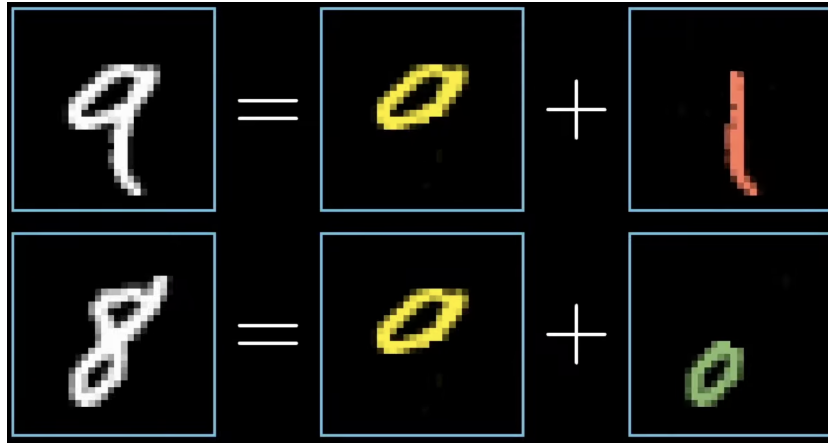


Figure 15: Sample Pattern Recognition of 8 and 9

Now, we aren't saying that the hidden layers will *actually* use these as methods for recognizing the numbers, but rather, this idea represents the basic intuition of how the hidden layers might filter numbers by characteristics.

Knowing this, how exactly does a node determine whether or not a certain pattern is seen in the input? It does this by using *weights and biases*. For example, a number '7', we might say, is characterized by having a straight edge at the top. What we can then do, is add a positive weight (positive multiplier) for all the input nodes that correspond to the likely places a straight edge might be located, and superimpose a negative weight on the edges of the line. This is illustrated below:

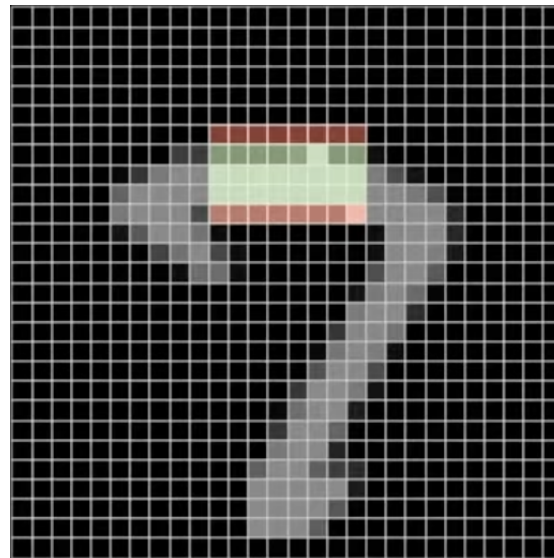


Figure 16: Detecting the Edge on top of 7

Similarly, the bias may be thought of as a threshold value that our node values need to surpass in order to register as an activated node. It may help to recall the activation function as being defined as:

$$\text{Activation} = f(w_1x_1 + w_2x_2 + \theta)$$

Where in this context, we define the activation function as the sigmoid function introduced previously:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Essentially, by continuously updating the weights and biases of each node, we are able to fine-tune the hidden layers to be able to recognize the patterns in the input data and determine a correct guess. In the next section, we go over the entire process for how we update these weights and biases in the process known as **Backpropagation**.

5.3 How to update the weights and biases?

On our first iteration of the neural network, we assign each node random weight and bias values. After this first iteration has completed, the output nodes will produce a set of outputs (O). Subsequently, these outputs can be compared to the target values for the problem. The difference between the output of the output node and the actual target value is called the **Error/loss**. The full process of neural network training is as follows:

1. Let the network calculate output with a given set of input nodes
2. calculate the **error/loss** function.
3. Update the weights and biases between layers (Backprop)
4. Repeat from step 1

Assuming the **activation function** is the **sigmoid function**, $\sigma(x) = \frac{1}{1+e^{-x}}$. The **error/loss function** is given by $E = \frac{1}{2} \sum_{\text{all } k} (O_k - T_k)^2$. Subsequently, the formulas for updating weights and biases are given as follows:

$$\begin{aligned}\delta_k &= (O_k - T_k)O_k(1 - O_k) \\ \delta_j &= O_j(1 - O_j) \sum_{i=1}^k \delta_k w_{jk}^k \\ w_{jk} &\leftarrow w_{jk} - \eta \delta_k O_j & \theta_k &\leftarrow \theta_k - \eta \delta_k \\ w_{ij} &\leftarrow w_{ij} - \eta \delta_j O_i & \theta_j &\leftarrow \theta_j - \eta \delta_j\end{aligned}$$

Where:

- O_i is the input in layer i
- O_j is the output of node in layer j
- O_k is the output of node in layer k
- T_k is the target output of node in layer k
- w_{ij} is the weight connecting node in layer i to node in layer j
- w_{jk} is the weight connecting node in layer j to node in layer k
- θ_j is the bias of node in layer j
- θ_k is the bias of node in layer k
- δ_k is the change of parameters between hidden and output layer, used to update w_{jk} and θ_k
- δ_j is the change of parameters between input and hidden layer, used to update w_{ij} and θ_j
- η is the learning rate.

Let us try to understand the equations above. Namely, the equations for δ_k and δ_j . We define δ_k as follows:

$$\delta_k = (O_k - T_k)O_k(1 - O_k)$$

How do we interpret this? To start, one way to think about δ_k is how much do we “blame” each output neuron. That is, for every neuron in the output layer, its personal “error signal” (δ_k) is defined by the product between:

- $(O_k - T_k)$, which is the **raw difference** of the output and target.
- $O_k(1 - O_k)$: This represents the **derivative of the Sigmoid Function**. Where the derivative of the sigmoid represents the **certainty** of the prediction of that neuron.

So essentially, the interpretation for δ_k is that it represents the error of the neuron multiplied by the certainty that neuron had in making its prediction. If the error is large but the certainty is high, then the neuron is less “malleable” compared to one that had less certainty, thus the neuron with less certainty is corrected more. Similarly, for δ_j , we define it as:

$$\delta_j = O_j(1 - O_j) \sum_{i=1}^k \delta_k w_{jk}^k$$

Initially, we may interpret the above equation as saying that for each neuron in the hidden layer, its error signal is calculated as the product between:

- $O_j(1 - O_j)$: This, again, is the derivative of the sigmoid function and is a representation of the neurons certainty.
- $\sum_{i=1}^k \delta_k w_{jk}^k$: This can be thought of as how much the error signal (blame) of each node in the output layer is distributed across each node in the hidden layer (**backpropagated blame**). δ_k represents the error signal of the output neuron, and w_{jk}^k represents the weight of the connection from the hidden neuron to every hidden output neron k . This is further illustrated in the figure below:

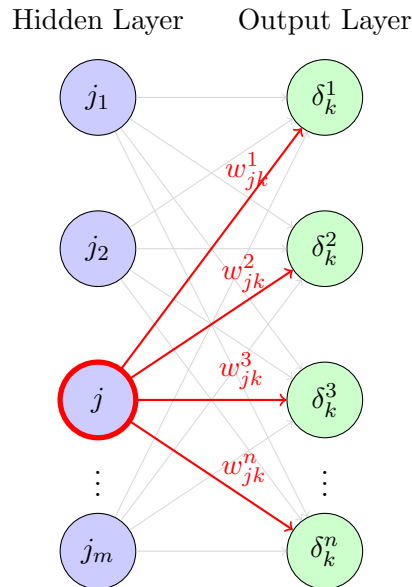


Figure 17: Error signal of hidden layers

Finally, note that when we are dealing with neural networks with multiple hidden layers, simply substitute δ_k in the original δ_j equation, with the δ_j values of the hidden layer after it.